

Project 19 - LLM Gateway Build Spec

Project 19 — Enterprise GenAI Project Catalog, BVRIT — Complete Build Specification

Prepared by S

August 2026

[Project 19 — LLM Gateway: Routing, Caching & Cost Control](#)

[Build Document Set — Index](#)

[Key decisions already made \(do not re-litigate these — they are locked\)](#)

[What “done” looks like](#)

[01 — Product Requirements Document](#)

[Project 19: LLM Gateway — Routing, Caching & Cost Control](#)

[Document Control](#)

[Executive Summary](#)

[02 — Architecture](#)

[Document Control](#)

[Executive Summary](#)

[03 — API Contracts](#)

[Document Control](#)

[Executive Summary](#)

[POST /v1/chat/completions](#)

[GET /v1/admin/teams](#)

[POST /v1/admin/teams](#)

[GET /healthz](#)

[Response metadata fields — precise definitions \(no ambiguity\)](#)

[04 — Build Plan](#)

[Document Control](#)

[Executive Summary](#)

[Phase 0 — Project scaffolding](#)

[Phase 1 — Data layer](#)

[Phase 2 — Provider clients](#)

[Phase 3 — Routing engine](#)

[Phase 4 — Caching](#)

[Phase 5 — Quota & rate limiting](#)

[Phase 6 — Cost attribution & logging](#)

[Phase 7 — Admin & health endpoints](#)

[Phase 8 — Dashboard](#)

[Phase 9 — Containerization & demo script](#)

[Phase 10 — Documentation & handoff](#)

[05 — Test Plan](#)

[Document Control](#)

[1. Purpose and scope](#)

[2. Test environment](#)

[3. Automated test suite](#)

[4. Manual QA scenarios \(run against live providers\)](#)

[5. Regression checklist \(run before any demo/presentation\)](#)

[6. Known limitations to disclose \(not defects\)](#)

Project 19 — LLM Gateway: Routing, Caching & Cost Control

Build Document Set — Index

This folder is the complete, unambiguous build spec for Project 19 from the BVRIT Enterprise GenAI Project Catalog. It is written to be handed directly to a coding agent (Claude Code, Cursor, Copilot Workspace, etc.) or followed manually. Read the documents in this order:

#	Document	Purpose
1	01_PRD.md	What we’re building and why. Scope, non-goals, KPIs.
2	02_ARCHITECTURE.md	System design, tech stack (exact versions), folder structure, data models, config files.
3	03_API_CONTRACTS.md	Every HTTP endpoint: method, path, request/response schema, status codes.
4	04_BUILD_PLAN.md	Phase-by-phase implementation plan with file-level tasks and acceptance criteria. Give this file to the coding agent as the primary task list.
5	05_TEST_PLAN.md	Concrete test cases (unit, integration, manual QA) mapped to the catalog’s required test scenarios.

Key decisions already made (do not re-litigate these — they are locked)

- **Language/runtime:** Python 3.11, FastAPI + Uvicorn.
- **Model providers (all free, no credit card):**
 - **Groq** (primary — fast, generous free tier: Llama 3.1/3.3 models).
 - **Google Gemini** (secondary — free tier via Google AI Studio: Gemini 2.0/2.5 Flash).
 - **OpenRouter free-tier models** (tertiary fallback pool — `:free` suffixed models, used only when both above fail).
 - GitHub Models is **not** used — it was fully retired by GitHub on July 30, 2026.
- **Cache:** SQLite-backed (exact-match + semantic via local embeddings). No Redis dependency required to run for free; Redis is an optional swap documented but not required.
- **Quotas/rate limits:** Per-team token budgets enforced via SQLite + FastAPI middleware.
- **Observability:** Lightweight custom Streamlit dashboard reading the same SQLite DB. No Langfuse/Prometheus/Grafana — those are noted as a “Stretch goal” only.
- **Deployment:** Docker Compose for local/dev. Free-tier hosting path documented (Render for the API, Streamlit Community Cloud for the dashboard) — no paid infrastructure required anywhere.
- **Auth:** Simple `X-Team-Key` header mapped to a `team_id` via a config table — sufficient for a student/hackathon-grade project, explicitly not enterprise SSO.

What “done” looks like

All 6 required architecture pieces from the catalog are implemented and demoable: 1. One internal endpoint fronting multiple backend models. 2. Task-based routing with automatic fallback. 3. Semantic + exact caching (visible cache hits in the dashboard). 4. Per-team quotas and rate limits (a team can be throttled). 5. A spend dashboard with anomaly alerts. 6. Cost attribution per team, visible in the dashboard.

If any of these six is missing, the build is not complete — do not mark the project done until all six are demonstrable per `05_TEST_PLAN.md`.

01 — Product Requirements Document

Project 19: LLM Gateway — Routing, Caching & Cost Control

Document Control

Field	Value
Project	Project 19 — LLM Gateway: Routing, Caching & Cost Control
Document	Product Requirements Document (PRD)
Version	1.0
Status	Approved for build
Related documents	02_ARCHITECTURE.md, 03_API_CONTRACTS.md, 04_BUILD_PLAN.md, 05_TEST_PLAN.md

Executive Summary

This project delivers a single internal gateway that every application calls instead of hitting LLM providers directly. It routes each request to the most appropriate free-tier model, falls back automatically if a provider fails, caches repeated or near-duplicate queries, enforces per-team usage budgets, and gives a live view of spend and usage patterns. The result: predictable cost, resilience to any one provider's outages or free-tier limits, and full visibility into who is spending what.

1. Problem statement

Multiple applications need to call LLMs. Without a shared gateway, each app hard-codes a provider/model, duplicates retry logic, has no idea what it's spending, and one runaway app can blow the whole budget. This project builds the single internal endpoint every app should call instead of hitting providers directly.

2. Goals

- G1. One stable internal HTTP endpoint (`POST /v1/chat/completions`) that fronts multiple backend LLM providers, OpenAI-compatible in shape.
- G2. Requests are routed to the cheapest model capable of handling them, with automatic fallback to another provider/model if the chosen one errors, times out, or is rate-limited.
- G3. Repeated or semantically similar requests are served from cache instead of hitting a provider again.
- G4. Each "team" (a caller identified by an API key) has a token budget; exceeding it throttles further requests and raises an alert.
- G5. A dashboard shows spend, request volume, cache-hit rate, and per-team cost breakdown, with anomaly alerts for unusual spend spikes.

3. Non-goals (explicitly out of scope)

- NG1. No enterprise SSO/OAuth — a static per-team API key is sufficient.
- NG2. No support for streaming responses in v1 (non-streaming `chat/completions` only). Streaming is a stretch goal, not required.
- NG3. No image/audio/multimodal input — text chat completions only.
- NG4. No multi-region or high-availability deployment — single instance is fine for this project's scale.
- NG5. No paid infrastructure of any kind. Every component must run for free (local Docker Compose, or free hosting tiers).

4. Target user

The platform/infra “team” in the scenario, and — from the caller’s perspective — any application that would otherwise call an LLM provider directly. In this project, the “applications” calling the gateway are simulated by a test harness (see `05_TEST_PLAN.md`) plus a simple demo chat client.

5. Success metrics (KPIs)

KPI	Target
Single endpoint adoption	100% of test traffic goes through the gateway, none direct to a provider
Cost reduction from caching	≥30% of a repeated-query test batch served from cache (cache hit, \$0 provider cost)
Routing correctness	Simple prompts route to the cheap/fast model ≥90% of the time in the test suite; complex prompts route to the stronger model ≥90% of the time
Fallback resilience	0 user-facing failures when the primary provider is forced to fail in tests
Quota enforcement	A team that exceeds its budget receives HTTP 429 on the very next request, not later
Spend visibility	Every logged request is attributable to a <code>team_id</code> , model, and cost in the dashboard within one refresh cycle

6. Functional requirements

- FR1 — Gateway endpoint.** `POST /v1/chat/completions` accepts an OpenAI-style request body (see `03_API_CONTRACTS.md`) and an `X-Team-Key` header identifying the caller.
- FR2 — Routing engine.** Classify each incoming request as `simple` or `complex` using a deterministic heuristic (no extra LLM call — must not itself cost money or add material latency). Route `simple` → cheap/fast model, `complex` → stronger model. Routing rules are defined in a config file, not hard-coded (see `routing_rules.yaml` in `02_ARCHITECTURE.md`).
- FR3 — Fallback.** If the routed provider/model call fails (HTTP error, timeout, or rate-limit response), retry once against the same provider with backoff, then fail over to the next provider in the configured chain. Only return an error to the caller if every provider in the chain fails.
- FR4 — Exact cache.** Before calling any provider, compute a normalized hash of `(model_tier, messages, temperature, max_tokens)`. If a non-expired cache entry exists, return it directly — no provider call.
- FR5 — Semantic cache.** If no exact match, embed the latest user message locally (no external API call) and compare against cached embeddings for the same `model_tier`. If cosine similarity ≥ configured threshold (default 0.92), return the cached response, tagged `cache: "semantic"` in the response metadata.
- FR6 — Quota enforcement.** Each team has a daily token budget (configurable). Before routing, check the team’s tokens used today. If the request would exceed budget, return HTTP 429 with a clear error body and do not call any provider.
- FR7 — Rate limiting.** Each team also has a requests-per-minute limit, enforced independently of the token budget.
- FR8 — Logging & cost attribution.** Every request (cache hit or provider call) is logged with: timestamp, `team_id`, route decision, provider, model, input/output tokens, computed cost, cache status, latency, and success/failure.
- FR9 — Dashboard.** A Streamlit app reads the log table and shows: total spend, spend by team (table + bar chart), spend by model, request volume over time, cache-hit rate, and an anomaly alert banner when a team’s spend rate in the current hour exceeds $N \times$ its 7-day average (default $N=3$).
- FR10 — Admin config.** Teams, their API keys, and their budgets are defined in a config file (or a `teams` table seeded from it) — no code change needed to add a team.

7. Constraints

- Every provider used must be genuinely free (no credit card required for the free tier used).
- The whole system must run locally via `docker compose up` with zero paid services.
- Must degrade gracefully: if a provider's free-tier daily quota is exhausted mid-demo, the fallback chain must route around it, not crash the gateway.

8. Assumptions

- The builder (S) has, or will create, free accounts and API keys for: Groq (<https://console.groq.com>), Google AI Studio / Gemini (<https://aistudio.google.com>), and OpenRouter (<https://openrouter.ai>).
- Local development happens on a machine that can run Docker Desktop (or Docker Engine on Linux) and Python 3.11.
- “Team” in this project is a synthetic concept for the demo — a handful of hardcoded team IDs (e.g. `team-sales`, `team-support`, `team-eng`) is sufficient; it does not need a real user-management system.

02 — Architecture

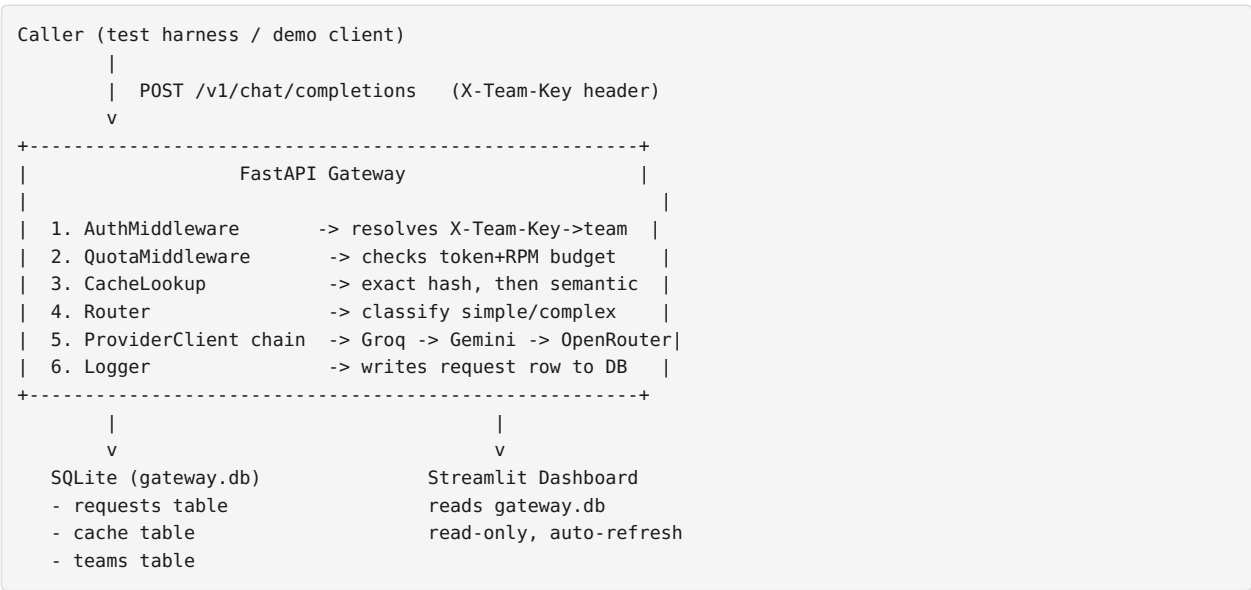
Document Control

Field	Value
Project	Project 19 — LLM Gateway: Routing, Caching & Cost Control
Document	System Architecture
Version	1.0
Status	Approved for build
Related documents	01_PRD.md, 03_API_CONTRACTS.md, 04_BUILD_PLAN.md, 05_TEST_PLAN.md

Executive Summary

The gateway is a FastAPI service backed by SQLite, sitting in front of three free-tier LLM providers (Groq, Google Gemini, OpenRouter). Every request passes through auth, quota checks, a two-tier cache (exact + semantic), a deterministic router with per-provider circuit breakers, and structured cost-attributed logging. A read-only Streamlit dashboard visualizes the resulting data. The entire stack runs for free via Docker Compose, with an optional path to free-tier cloud hosting.

1. High-level diagram (describe, since this is a text spec)



2. Tech stack (exact versions — pin these)

Component	Choice	Version
Language	Python	3.11
Web framework	FastAPI	0.115.x
ASGI server	Uvicorn	0.32.x
HTTP client (to providers)	httpx	0.27.x
Data validation	Pydantic	2.9.x
DB	SQLite via <code>sqlite3</code> (stdlib) +	—

	sqlmodel 0.0.22 for ORM convenience	
Embeddings (semantic cache)	sentence-transformers 3.1.x, model all-MiniLM-L6-v2 (runs locally, ~80MB, no API cost)	—
Similarity	numpy 2.1.x, cosine similarity, brute-force (fine at this scale — hundreds to low thousands of cache rows)	—
Dashboard	Streamlit 1.39.x + pandas 2.2.x + plotly 5.24.x	—
Config	YAML via pyyaml 6.0.x	—
Testing	pytest 8.3.x, pytest-asyncio 0.24.x, respx 0.21.x (mock httpx calls)	—
Containerization	Docker + Docker Compose v2	—

3. Repository / folder structure

```

llm-gateway/
├── docker-compose.yml
├── .env.example
├── requirements.txt
├── gateway/
│   ├── main.py                # FastAPI app entrypoint
│   ├── config.py              # loads .env + routing_rules.yaml + teams.yaml
│   ├── db.py                  # SQLAlchemy engine/session, init_db()
│   ├── models.py              # SQLAlchemy table classes: Team, RequestLog, CacheEntry
│   ├── schemas.py             # Pydantic request/response models (API contracts)
│   ├── auth.py                # resolve X-Team-Key -> Team
│   ├── quota.py               # token budget + RPM checks, raises QuotaExceeded
│   ├── router.py              # classify_request(), pick_route()
│   ├── cache.py               # exact_lookup(), semantic_lookup(), store()
│   ├── embeddings.py          # load model once, embed(text) -> np.ndarray
│   ├── providers/
│   │   ├── base.py            # ProviderClient ABC: complete(request) -> ProviderResponse
│   │   ├── groq_client.py
│   │   ├── gemini_client.py
│   │   ├── openrouter_client.py
│   │   └── circuit_breaker.py # per-provider failure tracking
│   ├── cost.py                # token counting + pricing table -> cost in USD
│   ├── logging_service.py     # write RequestLog rows
│   └── routes/
│       ├── chat.py            # POST /v1/chat/completions
│       ├── admin.py           # GET /v1/admin/teams, POST /v1/admin/teams
│       └── health.py          # GET /healthz
├── config/
│   ├── routing_rules.yaml
│   ├── teams.yaml
│   └── pricing.yaml
├── dashboard/
│   └── app.py                  # Streamlit app
├── tests/
│   ├── conftest.py
│   ├── test_routing.py
│   ├── test_cache.py
│   ├── test_quota.py
│   ├── test_fallback.py
│   └── test_cost_attribution.py
└── data/
    └── gateway.db              # created at runtime, gitignored

```

4. Data models (SQLModel table definitions — implement exactly)

```
# gateway/models.py
```

```

class Team(SQLModel, table=True):
    id: str = Field(primary_key=True)          # e.g. "team-sales"
    name: str
    api_key: str = Field(unique=True, index=True)
    daily_token_budget: int = 200_000
    rpm_limit: int = 20
    created_at: datetime = Field(default_factory=datetime.utcnow)

class RequestLog(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    timestamp: datetime = Field(default_factory=datetime.utcnow, index=True)
    team_id: str = Field(index=True)
    route_decision: str          # "simple" | "complex"
    provider: str                # "groq" | "gemini" | "openrouter" | "cache"
    model: str
    cache_status: str            # "miss" | "exact" | "semantic"
    input_tokens: int
    output_tokens: int
    cost_usd: float
    latency_ms: int
    success: bool
    error_message: Optional[str] = None

class CacheEntry(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    cache_key: str = Field(index=True)          # sha256 hash for exact match
    model_tier: str          # "simple" | "complex"
    prompt_text: str          # normalized latest user message
    embedding: bytes          # np.ndarray.tobytes()
    response_json: str        # serialized full response
    created_at: datetime = Field(default_factory=datetime.utcnow)
    expires_at: datetime
    hit_count: int = 0

```

5. Config files (exact formats — the coding agent must produce files matching these schemas)

`config/teams.yaml` — seeds the `Team` table on startup if empty:

```

teams:
  - id: team-sales
    name: Sales Team
    api_key: sk-team-sales-demo-key
    daily_token_budget: 100000
    rpm_limit: 15
  - id: team-support
    name: Support Team
    api_key: sk-team-support-demo-key
    daily_token_budget: 150000
    rpm_limit: 20
  - id: team-eng
    name: Engineering Team
    api_key: sk-team-eng-demo-key
    daily_token_budget: 50000
    rpm_limit: 10

```

`config/routing_rules.yaml` — defines the classifier thresholds and the provider chain per tier:

```

classifier:
  # A request is "complex" if EITHER condition is true; else "simple".
  max_simple_chars: 400          # prompt longer than this -> complex
  complex_keywords:              # presence of any (case-insensitive) -> complex
    - "step by step"
    - "analyze"
    - "architecture"
    - "code review"

```

```

- "compare"
- "explain in detail"
- "write a"
- "debug"

routes:
  simple:
    chain:
      - provider: groq
        model: llama-3.1-8b-instant
      - provider: gemini
        model: gemini-2.0-flash
      - provider: openrouter
        model: meta-llama/llama-3.1-8b-instruct:free
    complex:
      chain:
        - provider: groq
          model: llama-3.3-70b-versatile
        - provider: gemini
          model: gemini-2.0-flash
        - provider: openrouter
          model: deepseek/deepseek-chat:free

cache:
  semantic_similarity_threshold: 0.92
  ttl_seconds: 86400 # 24h

circuit_breaker:
  failure_threshold: 3 # consecutive failures before opening
  cooldown_seconds: 60 # time before retrying an open circuit

```

config/pricing.yaml — used purely for cost *attribution/display* since providers are free; this lets the dashboard show “notional cost saved” as if these were paid API calls (useful for the KPI in the PRD, and realistic for when a team later swaps in paid providers):

```

# USD per 1M tokens, input/output – figures are illustrative published
# list-price figures for the underlying open model classes, used only to
# compute a notional cost for the dashboard.
pricing:
  groq/llama-3.1-8b-instant: { input: 0.05, output: 0.08 }
  groq/llama-3.3-70b-versatile: { input: 0.59, output: 0.79 }
  gemini/gemini-2.0-flash: { input: 0.10, output: 0.40 }
  openrouter/meta-llama/llama-3.1-8b-instruct:free: { input: 0.0, output: 0.0 }
  openrouter/deepseek/deepseek-chat:free: { input: 0.0, output: 0.0 }

```

.env.example :

```

GROQ_API_KEY=
GEMINI_API_KEY=
OPENROUTER_API_KEY=
GATEWAY_DB_PATH=./data/gateway.db
LOG_LEVEL=INFO

```

6. Provider client interface (implement this ABC, one subclass per provider)

```

# gateway/providers/base.py
class ProviderResponse(BaseModel):
    text: str
    input_tokens: int
    output_tokens: int
    raw: dict

class ProviderClient(ABC):
    name: str
    @abstractmethod
    async def complete(self, model: str, messages: list[dict],

```

```
temperature: float, max_tokens: int) -> ProviderResponse:
    """Raises ProviderError on any failure (HTTP error, timeout, rate limit)."""
```

- `groq_client.py` calls `https://api.groq.com/openai/v1/chat/completions` (OpenAI-compatible; use the `openai` SDK pointed at Groq's `base_url`, or raw `httpx`).
- `gemini_client.py` calls the Gemini API (`https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent`) with `key=` query param from `GEMINI_API_KEY`; map the OpenAI-style `messages` list to Gemini's `contents` format inside this client only — the rest of the system never sees Gemini's native schema.
- `openrouter_client.py` calls `https://openrouter.ai/api/v1/chat/completions` (OpenAI-compatible) with `OPENROUTER_API_KEY`.
- Every client must set an 8-second `httpx` timeout and raise `ProviderError` (defined in `base.py`) on timeout, non-2xx response, or malformed body — never let a raw exception escape the client.

7. Circuit breaker (per provider, in-memory, process-local)

Simple state machine per provider name, no external dependency: - States: `closed` (normal), `open` (skip this provider), `half_open` (single trial request allowed). - On `ProviderError`: increment failure count. If count $\geq$ `failure_threshold` (config), transition to `open`, record `opened_at`. - While `open`: if `now - opened_at > cooldown_seconds`, transition to `half_open` and allow exactly one request through. - `half_open` success $\rightarrow$ `closed`, reset failure count. `half_open` failure $\rightarrow$ back to `open`, reset `opened_at`. - The router must skip any provider currently `open` and go to the next in the chain, without waiting for its timeout.

8. Routing algorithm (deterministic, implement exactly)

```
def classify(messages, rules) -> "simple" | "complex":
    last_user_msg = last message with role == "user"
    if len(last_user_msg.content) > rules.classifier.max_simple_chars:
        return "complex"
    lowered = last_user_msg.content.lower()
    if any(kw in lowered for kw in rules.classifier.complex_keywords):
        return "complex"
    return "simple"
```

9. Cache key & semantic lookup (implement exactly)

- Exact key: `sha256(json.dumps({model_tier, messages, temperature, max_tokens}, sort_keys=True))`.
- Semantic lookup: only runs on exact-cache miss. Embed the last user message with `all-MiniLM-L6-v2`. Compare against all non-expired `CacheEntry` rows with the same `model_tier` using cosine similarity. If max similarity $\geq$ `cache.semantic_similarity_threshold`, return that entry's `response_json` and increment its `hit_count`.
- On any provider response (cache miss path), write a new `CacheEntry` with `expires_at = now + cache.ttl_seconds`.

10. Quota & rate limit checks (order of operations in `chat.py`)

1. Resolve team from `X-Team-Key` (401 if unknown key).
2. Check RPM: count `RequestLog` rows for this team in the last 60 seconds; if $\geq$ `rpm_limit`, return 429 `{"error": "rate_limited"}`.
3. Check daily token budget: sum `input_tokens + output_tokens` for this team since local midnight UTC; if $\geq$ `daily_token_budget`, return 429 `{"error": "quota_exceeded"}`.
4. Only after both checks pass: run cache lookup, then routing.
5. Log the request (success or failure) regardless of outcome, including 429s (with `success=false`, `error_message` set), so the dashboard can show throttling events.

11. Dashboard data contract

The Streamlit app (`dashboard/app.py`) opens `data/gateway.db` **read-only** and queries `RequestLog` directly (no separate API needed — same-container or same-volume access via Docker Compose). Panels required: 1.

Total spend (sum `cost_usd`) — big number, today vs. all-time. 2. Spend by team — bar chart. 3. Spend by model — bar chart. 4. Request volume over time — line chart, hourly buckets. 5. Cache hit rate — $(\text{exact+semantic}) / \text{total requests}$, as a percentage. 6. Anomaly banner: for each team, compute current-hour spend vs. average hourly spend over the last 7 days; if current > 3× average AND current-hour request count ≥ 5 (avoid false alarms on tiny samples), show a red banner: "⚠ {team} spending {X}x above normal this hour."

12. Deployment

Local (required, must work with zero config beyond `.env`):

```
docker compose up --build
```

`docker-compose.yml` defines two services: `gateway` (port 8000) and `dashboard` (port 8501), both mounting `./data` as a shared volume so the dashboard reads what the gateway writes.

Free-tier hosting (documented, optional for the demo): - Gateway → Render.com free Web Service (Dockerfile deploy). Note: free tier sleeps after inactivity — document this as a known limitation, not a bug. - Dashboard → Streamlit Community Cloud (free), pointed at a small Postgres instead of SQLite if the dashboard needs to run in a separate process from the gateway with no shared filesystem — use Supabase's free Postgres tier for this case. **This swap is optional**; local Docker Compose with SQLite is the primary, required deployment target for grading/demo purposes.

03 — API Contracts

Document Control

Field	Value
Project	Project 19 — LLM Gateway: Routing, Caching & Cost Control
Document	API Contracts
Version	1.0
Status	Approved for build
Related documents	02_ARCHITECTURE.md, 04_BUILD_PLAN.md, 05_TEST_PLAN.md

Executive Summary

Four endpoints make up the entire public surface: the core chat-completion gateway, two lightweight admin endpoints for team management, and a health check. The core endpoint is deliberately OpenAI-compatible so it can be adopted with a one-line `base_url` change in any existing client.

All endpoints are served by the FastAPI app at base URL `http://localhost:8000`.

POST /v1/chat/completions

The core gateway endpoint. OpenAI-compatible shape so any existing OpenAI client can point at this with a different `base_url`.

Headers | Header | Required | Description | |—|—|—| | `X-Team-Key` | Yes | Team's API key, defined in `config/teams.yaml` | | `Content-Type` | Yes | `application/json` |

Request body

```
{
  "messages": [
    {"role": "user", "content": "Summarize this changelog in 3 bullets: ..."}
  ],
  "temperature": 0.7,
  "max_tokens": 512
}
```

Field	Type	Required	Notes
<code>messages</code>	array of <code>{role, content}</code>	Yes	<code>role</code> ∈ <code>system, user, assistant</code> . Must contain at least one <code>user</code> message.
<code>temperature</code>	float	No	Default <code>0.7</code> . Range <code>0.0–2.0</code> .
<code>max_tokens</code>	int	No	Default <code>512</code> . Max <code>4096</code> .

Success response — 200

```
{
  "id": "req_a1b2c3",
  "route_decision": "simple",
}
```

```

    "provider": "groq",
    "model": "llama-3.1-8b-instant",
    "cache_status": "miss",
    "message": {"role": "assistant", "content": "..."},
    "usage": {
      "input_tokens": 142,
      "output_tokens": 87,
      "cost_usd": 0.0000178
    },
    "latency_ms": 612
  }

```

Error responses | Status | Body | Trigger | —|—|—| 401 | {"error": "invalid_api_key"} | Missing/unknown X-Team-Key | | 400 | {"error": "invalid_request", "detail": "..."} | Malformed body (Pydantic validation failure) | | 429 | {"error": "rate_limited", "retry_after_seconds": 12} | RPM limit exceeded | | 429 | {"error": "quota_exceeded", "reset_at": "2026-08-26T00:00:00Z"} | Daily token budget exceeded | | 502 | {"error": "all_providers_failed", "detail": "..."} | Every provider in the fallback chain failed | | 504 | {"error": "timeout"} | All providers timed out |

GET /v1/admin/teams

Lists all configured teams and their current usage (no secrets exposed).

Headers: none required for this project's scope (no admin auth — note this explicitly as a known simplification, not a gap to silently ignore).

Response — 200

```

{
  "teams": [
    {
      "id": "team-sales",
      "name": "Sales Team",
      "daily_token_budget": 100000,
      "tokens_used_today": 23421,
      "rpm_limit": 15
    }
  ]
}

```

POST /v1/admin/teams

Adds a new team at runtime (writes to the `Team` table; does not persist back to `teams.yaml`).

Request body

```

{
  "id": "team-marketing",
  "name": "Marketing Team",
  "api_key": "sk-team-marketing-demo-key",
  "daily_token_budget": 75000,
  "rpm_limit": 10
}

```

Response — 201 the created team object (same shape as GET, `tokens_used_today: 0`). **Response — 409** {"error": "team_id_exists"} if `id` already present.

GET /healthz

Liveness check for Docker/hosting platforms.

Response — 200

```
{"status": "ok", "providers": {"groq": "closed", "gemini": "closed", "openrouter": "closed"}}
```

`providers` reflects each circuit breaker's current state (`closed` / `open` / `half_open`), useful for the dashboard's provider-health panel and for debugging during the demo.

Response metadata fields — precise definitions (no ambiguity)

- `route_decision`: `"simple"` or `"complex"`, the classifier's output — always present, even on a cache hit (the tier that was looked up).
- `provider`: which provider actually served the response. Value is `"cache"` when served from cache (not the original provider that produced the cached response); the dashboard's per-provider spend chart should therefore use `cost_usd` (which is `0` on cache hits) rather than `provider` alone to compute savings.
- `cache_status`: `"miss"`, `"exact"`, or `"semantic"`.
- `usage.cost_usd`: computed from `config/pricing.yaml`; `0.0` for any `openrouter/*:free` model and for all cache hits.

04 — Build Plan

Document Control

Field	Value
Project	Project 19 — LLM Gateway: Routing, Caching & Cost Control
Document	Implementation / Build Plan
Version	1.0
Status	Approved for build
Related documents	01_PRD.md, 02_ARCHITECTURE.md, 03_API_CONTRACTS.md, 05_TEST_PLAN.md

Executive Summary

Eleven sequential phases take the project from an empty repository to a containerized, dashboarded, fully tested gateway. Each phase lists its concrete tasks, the exact files it produces, and the acceptance criteria that must pass before the next phase begins. This document is written to be handed directly to a coding agent as its task list.

Work through phases in order. Do not start a phase until the previous phase's acceptance criteria pass. After each phase, run the relevant tests from `05_TEST_PLAN.md` before moving on.

Phase 0 — Project scaffolding

Tasks 1. Create the folder structure exactly as in `02_ARCHITECTURE.md` §3. 2. Write `requirements.txt` pinning the versions in `02_ARCHITECTURE.md` §2. 3. Write `.env.example` as specified. Do not commit a real `.env`. 4. Write `config/teams.yaml`, `config/routing_rules.yaml`, `config/pricing.yaml` exactly as specified in `02_ARCHITECTURE.md` §5. 5. Initialize a git repo, add `.gitignore` covering `data/`, `.env`, `__pycache__`, `/*.db`.

Acceptance criteria - `pip install -r requirements.txt` succeeds in a clean venv. - Folder tree matches the spec exactly (no extra top-level dirs).

Phase 1 — Data layer

Tasks 1. Implement `gateway/models.py` with `Team`, `RequestLog`, `CacheEntry` exactly as defined in `02_ARCHITECTURE.md` §4. 2. Implement `gateway/db.py`: creates the SQLite engine at `GATEWAY_DB_PATH`, `init_db()` creates tables if missing, and seeds `Team` rows from `config/teams.yaml` if the `Team` table is empty. 3. Write `tests/conftest.py` with a pytest fixture that creates a fresh in-memory SQLite DB per test (`sqlite://`), overriding `GATEWAY_DB_PATH`.

Acceptance criteria - Running the app once creates `data/gateway.db` with 3 seeded teams matching `config/teams.yaml`. - `pytest tests/conftest.py` collects with no errors (fixture-only file, no tests yet, just confirm no import errors).

Phase 2 — Provider clients

Tasks 1. Implement `gateway/providers/base.py` (`ProviderClient`, `ABC`, `ProviderResponse`, `ProviderError`) exactly as in `02_ARCHITECTURE.md` §6. 2. Implement `groq_client.py`, `gemini_client.py`, `openrouter_client.py` per §6.

Each reads its API key from environment variables loaded via `gateway/config.py`. 3. Implement `gateway/providers/circuit_breaker.py` per §7 — a `CircuitBreaker` class with `record_success()`, `record_failure()`, `is_open()` -> `bool`, keyed per provider name, held in a module-level dict (process memory is fine — no persistence needed across restarts). 4. Write a small manual smoke-test script `scripts/smoke_test_providers.py` that sends “Say hello in exactly 3 words.” to each of the 3 providers directly (bypassing the gateway) using real API keys from `.env`, and prints the response + latency for each. This is for the human builder to confirm their API keys work — not part of the automated test suite.

Acceptance criteria - `python scripts/smoke_test_providers.py` (with real keys in `.env`) prints a successful response from all 3 providers. - `tests/test_fallback.py` (write now, per `05_TEST_PLAN.md §Fallback`) passes using `respx`-mocked HTTP calls — no real API keys needed for automated tests.

Phase 3 — Routing engine

Tasks 1. Implement `gateway/config.py` to load and validate `routing_rules.yaml` into a Pydantic settings object. 2. Implement `gateway/router.py`: - `classify(messages, rules)` -> `Literal["simple", "complex"]` per `02_ARCHITECTURE.md §8`. - `get_chain(tier, rules)` -> `list[RouteStep]` returns the ordered provider/model chain for a tier from `routing_rules.yaml`. - `async def route_request(messages, temperature, max_tokens, rules, provider_clients)` -> `tuple[ProviderResponse, str provider_used, str model_used]`: classifies, gets the chain, iterates the chain skipping any provider whose circuit breaker `is_open()`, calls `client.complete(...)`, and on `ProviderError` records the failure on that provider’s breaker and tries the next step. Raises `AllProvidersFailedError` if the whole chain is exhausted.

Acceptance criteria - `tests/test_routing.py` passes: a short/simple prompt classifies “simple”; a long prompt (>400 chars) and a prompt containing “analyze” both classify “complex”.

Phase 4 — Caching

Tasks 1. Implement `gateway/embeddings.py`: loads `all-MiniLM-L6-v2` once at module import (not per-request — this is the single most important performance detail in this phase), exposes `embed(text: str)` -> `np.ndarray`. 2. Implement `gateway/cache.py`: - `make_cache_key(model_tier, messages, temperature, max_tokens)` -> `str` (sha256 per §9). - `exact_lookup(session, key)` -> `CacheEntry | None`. - `semantic_lookup(session, model_tier, query_text, threshold)` -> `CacheEntry | None` — brute-force cosine similarity over all non-expired rows for that tier. - `store(session, key, model_tier, prompt_text, embedding, response_json, ttl_seconds)`. 3. Wire cache lookup into the request flow in `routes/chat.py` **before** routing: exact lookup first, then semantic lookup, then only if both miss, call `route_request`.

Acceptance criteria - `tests/test_cache.py` passes: identical request twice → second call is `cache_status: "exact"`, provider client is not called (assert via mock call count). A paraphrased near-duplicate request → third call is `cache_status: "semantic"`.

Phase 5 — Quota & rate limiting

Tasks 1. Implement `gateway/quota.py`: - `check_rpm(session, team)` -> `None`, raises `RateLimitedError` per §10 step 2. - `check_daily_budget(session, team)` -> `None`, raises `QuotaExceededError` per §10 step 3. 2. Implement `gateway/auth.py`: `resolve_team(session, api_key)` -> `Team`, raises `InvalidApiKeyError` if not found. 3. Wire both into `routes/chat.py` in the exact order specified in §10.

Acceptance criteria - `tests/test_quota.py` passes: a team with a tiny budget (set in test fixture) gets a normal 200 on request 1, then 429 `quota_exceeded` on a subsequent request that would exceed budget — no provider call is made for the rejected request (assert mock call count unchanged). - A separate test: 21 rapid requests from a team with `rpm_limit: 20` results in the 21st returning 429 `rate_limited`.

Phase 6 — Cost attribution & logging

Tasks 1. Implement `gateway/cost.py`: `compute_cost(provider, model, input_tokens, output_tokens, pricing_config) -> float`, reading `config/pricing.yaml`. Return `0.0` for any model/provider not found in the pricing table (log a warning, don't crash). 2. Implement `gateway/logging_service.py`: `log_request(session, ...)` -> `RequestLog`, called on every code path in `routes/chat.py` — success, cache hit, 429, and 502/504 — so nothing is invisible to the dashboard. 3. Wire `POST /v1/chat/completions` end-to-end per the flow in `03_API_CONTRACTS.md`.

Acceptance criteria - `tests/test_cost_attribution.py` passes: a mocked Groq response with known input/output token counts produces the exact expected `cost_usd` per the pricing table's per-token math. - Manually hit the endpoint 5 times with `curl` (using a real or mocked key) and confirm 5 rows appear in `RequestLog` via a quick `sqlite3 data/gateway.db "select * from requestlog;"`.

Phase 7 — Admin & health endpoints

Tasks 1. Implement `routes/admin.py`: `GET /v1/admin/teams`, `POST /v1/admin/teams` per `03_API_CONTRACTS.md`. 2. Implement `routes/health.py`: `GET /healthz`, including each circuit breaker's current state.

Acceptance criteria - `curl localhost:8000/healthz` returns 200 with all 3 providers "closed" on a fresh start. - `curl localhost:8000/v1/admin/teams` lists the 3 seeded teams with `tokens_used_today` matching the sum in `RequestLog`.

Phase 8 — Dashboard

Tasks 1. Implement `dashboard/app.py` per `02_ARCHITECTURE.md` §11 — all 6 panels. Use `st.cache_data(ttl=10)` on the DB-read function so the dashboard auto-refreshes every 10 seconds without hammering SQLite. 2. Add a sidebar filter: date range + team multiselect, applied to all panels.

Acceptance criteria - `streamlit run dashboard/app.py` (or via Docker Compose) shows real data after running the smoke tests / demo traffic script (Phase 9). - Manually force a "spend spike" (see `05_TEST_PLAN.md` §Anomaly) and confirm the red banner appears.

Phase 9 — Containerization & demo script

Tasks 1. Write `Dockerfile` for the gateway (multi-stage: install deps, copy `gateway/`, `config/`; `CMD` `["uvicorn", "gateway.main:app", "--host", "0.0.0.0", "--port", "8000"]`). 2. Write a second `Dockerfile` (or reuse with a different `CMD`) for the dashboard: `CMD ["streamlit", "run", "dashboard/app.py", "--server.address=0.0.0.0"]`. 3. Write `docker-compose.yml` per `02_ARCHITECTURE.md` §12 — two services, shared `./data` volume, gateway reads `.env`. 4. Write `scripts/demo_traffic.py`: sends a mixed batch of ~40 requests across the 3 seeded teams — a mix of simple/complex prompts, some exact duplicates, some paraphrased near-duplicates, and enough volume from one team to trip its quota — so a fresh reviewer can run one script and immediately see all 6 required capabilities lit up in the dashboard.

Acceptance criteria - `docker compose up --build` starts both services; gateway healthy at `:8000/healthz`, dashboard reachable at `:8501`. - `python scripts/demo_traffic.py` run against the running gateway populates the dashboard with visible: cache hits, at least one 429, spend by team/model, and (if you set a low anomaly threshold for the demo) the anomaly banner.

Phase 10 — Documentation & handoff

Tasks 1. Write `README.md` at the repo root (separate from this doc set): setup instructions, how to get free

API keys for Groq/Gemini/OpenRouter, how to run locally, how to run the demo script, screenshot of the dashboard. 2. Note the free-tier hosting option from `02_ARCHITECTURE.md` §12 as an optional “Deploy it” section.

Acceptance criteria - A reviewer who has never seen the project can clone the repo, follow `README.md`, and get the demo running in under 15 minutes.

05 — Test Plan

Document Control

Field	Value
Project	Project 19 — LLM Gateway: Routing, Caching & Cost Control
Document	Test Plan
Version	1.0
Status	Approved for build
Related documents	01_PRD.md, 02_ARCHITECTURE.md, 03_API_CONTRACTS.md, 04_BUILD_PLAN.md

1. Purpose and scope

This document defines the concrete tests that prove every functional requirement in `01_PRD.md` and every capability required by the catalog scenario is implemented and working. It covers three layers:

- 1. **Automated unit/integration tests** (`pytest` , provider calls mocked with `respx` — no real API keys or cost involved).
- 2. **Manual QA scenarios** (run against the live system with real, free-tier API keys) — these are what a reviewer or the builder runs before calling the project “done.”
- 3. **Demo script** (`scripts/demo_traffic.py`) — the single command that proves all six required capabilities in one pass, for a live walkthrough or presentation.

A build is not complete until every test in Section 3 passes and every scenario in Section 4 has been manually verified at least once against real providers.

2. Test environment

Item	Value
Automated tests	Run via <code>pytest</code> from repo root; use in-memory SQLite (<code>sqlite://</code>); all provider HTTP calls intercepted by <code>respx</code> — zero cost, zero network dependency, safe for CI.
Manual QA	Run against <code>docker compose up --build</code> ; requires real <code>GROQ_API_KEY</code> and <code>GEMINI_API_KEY</code> in <code>.env</code> (both free, no card). <code>OPENROUTER_API_KEY</code> optional — only needed to test the tertiary fallback path.
Test data	Fixture teams: <code>team-sales</code> , <code>team-support</code> , <code>team-eng</code> as seeded in <code>config/teams.yaml</code> .

3. Automated test suite

3.1 `tests/test_routing.py` — Routing correctness

ID	Case	Input	Expected
RT-01	Short, plain prompt	"What's the capital of	<code>classify()</code> -> "simple"

		France?"	
RT-02	Long prompt (>400 chars)	A 500-character prompt with no keywords	<code>classify()</code> -> "complex"
RT-03	Keyword trigger	"Can you analyze this dataset?" (short)	<code>classify()</code> -> "complex"
RT-04	Chain lookup — simple tier	tier= "simple"	Chain returned is exactly [groq/llama-3.1-8b-instant, gemini/gemini-2.0-flash, openrouter/...:free] in that order, matching <code>routing_rules.yaml</code>
RT-05	Chain lookup — complex tier	tier= "complex"	Chain returned starts with groq/llama-3.3-70b-versatile

3.2 tests/test_fallback.py — Provider fallback & circuit breaker

ID	Case	Setup	Expected
FB-01	Primary succeeds	Mock Groq 200	Response comes from groq ; Gemini/OpenRouter never called (assert mock call counts = 0)
FB-02	Primary fails once, retries, still fails, falls over	Mock Groq to return 500 both attempts, Gemini 200	Response comes from gemini ; Groq called exactly twice (1 try + 1 retry), Gemini called once
FB-03	Primary and secondary fail, tertiary succeeds	Mock Groq 500, Gemini 500, OpenRouter 200	Response comes from openrouter
FB-04	All providers fail	Mock all three to 500	Endpoint returns HTTP 502 all_providers_failed ; response is never silently empty
FB-05	Circuit breaker opens	Force 3 consecutive ProviderErrors from Groq (below failure_threshold)	4th request skips Groq entirely — Groq mock is not called; request goes straight to Gemini
FB-06	Circuit breaker half-open recovery	After FB-05, advance mocked clock past cooldown_seconds , mock Groq to succeed	Next request calls Groq (half-open trial), succeeds, breaker returns to closed
FB-07	Timeout handling	Mock Groq to hang past 8s	Request fails over to Gemini within a bounded total time (assert < 9.5s), not a hang

3.3 tests/test_cache.py — Caching

ID	Case	Setup	Expected
CA-01	Exact cache miss then hit	Send identical request twice	1st: cache_status: "miss", provider called once. 2nd: cache_status: "exact", provider call count unchanged (not called again)
CA-02	Semantic cache hit	Send "Summarize the Q3	2nd request: cache_status:

		earnings report", then a paraphrase "Give me a summary of the Q3 earnings report"	"semantic", no provider call
CA-03	Below similarity threshold	Send two clearly unrelated prompts in the same tier	2nd request: <code>cache_status: "miss"</code> (semantic lookup does not false-positive)
CA-04	TTL expiry	Insert a <code>CacheEntry</code> with <code>expires_at</code> in the past, then request the same prompt	<code>cache_status: "miss"</code> — expired entries are not returned
CA-05	Cache is tier-scoped	Same prompt text sent once classified "simple" and once forced "complex"	No cross-tier cache hit — each tier caches independently

3.4 `tests/test_quota.py` — Quotas & rate limiting

ID	Case	Setup	Expected
QT-01	Under budget	Team with <code>daily_token_budget: 10000</code> , 2000 tokens used so far	Request proceeds normally (200)
QT-02	Over budget	Same team, next request would push usage over 10000	HTTP 429 <code>quota_exceeded</code> ; provider mock call count unchanged (no call made)
QT-03	RPM limit	Team with <code>rpm_limit: 5</code> ; send 6 requests within 60 seconds	6th request returns HTTP 429 <code>rate_limited</code> ; <code>retry_after_seconds</code> present and > 0
QT-04	RPM window rolls over	After QT-03, wait (or mock clock past) 60 seconds	Next request succeeds (200)
QT-05	Invalid API key	Request with <code>X-Team-Key: does-not-exist</code>	HTTP 401 <code>invalid_api_key</code> ; no DB row written for a team, but the failed attempt is still logged (with <code>team_id: null</code> or a sentinel) for audit purposes
QT-06	429s are logged	Trigger QT-02	A <code>RequestLog</code> row exists for the rejected request with <code>success: false</code> , <code>error_message</code> set — visible in the dashboard's request volume, not silently dropped

3.5 `tests/test_cost_attribution.py` — Cost calculation

ID	Case	Setup	Expected
CO-01	Known token counts, priced model	Mock Groq response: 1000 input / 500 output tokens, model <code>llama-3.1-8b-instant</code>	<code>cost_usd == (1000/1_000_000)*0.05 + (500/1_000_000)*0.08</code> exactly
CO-02	Free model	Response from <code>openrouter/...:free</code>	<code>cost_usd == 0.0</code>
CO-03	Cache hit	Response served from	<code>cost_usd == 0.0</code> ,

		cache	<code>input_tokens / output_tokens</code> still reflect the original cached values (for dashboard token-volume accuracy)
CO-04	Unknown model in pricing table	Response from a model not listed in <code>pricing.yaml</code>	<code>cost_usd == 0.0</code> , a warning is logged, request does not fail

4. Manual QA scenarios (run against live providers)

These map directly to the catalog's six required capabilities. Run each once against the real running system and record pass/fail.

#	Scenario	Steps	Pass criteria
1	Single endpoint fronts multiple providers	Send 5 varied requests via <code>curl</code> to <code>/v1/chat/completions</code>	Response <code>provider</code> field shows at least 2 different providers across the batch (routing is actually happening, not hardcoded)
2	Task-based routing + fallback	Send one short factual prompt and one long "analyze this in detail..." prompt	Short prompt routes "simple" tier, long one routes "complex" tier — visible in response <code>route_decision</code>
2b	Fallback under real failure	Temporarily set an invalid <code>GROQ_API_KEY</code> in <code>.env</code> , restart, send a request	Request still succeeds, served by Gemini; <code>provider: "gemini"</code> in response
3	Caching	Send the exact same request twice, then a paraphrased version a third time	2nd response <code>cache_status: "exact"</code> ; 3rd response <code>cache_status: "semantic"</code> ; both near-instant (<100ms) vs. the ~500-1500ms of the first call
4	Quotas & rate limits	Temporarily set <code>team-eng</code> 's <code>rpm_limit: 3</code> in <code>teams.yaml</code> , restart, send 4 rapid requests	4th request returns 429
5	Spend dashboard + anomaly alert	Run <code>scripts/demo_traffic.py</code> , then open the dashboard	Spend-by-team and spend-by-model charts populated; cache-hit-rate panel shows a non-zero percentage
5b	Anomaly alert trigger	Manually send 10 rapid requests from one team (enough to be 3x its recent hourly average)	Red anomaly banner appears in the dashboard for that team within one refresh cycle (≤10s)
6	Cost attribution	Open the dashboard's spend-by-team panel after running the demo script	Every team that made requests shows a non-negative <code>cost_usd</code> total; sum of per-team costs equals the dashboard's total spend figure

5. Regression checklist (run before any demo/presentation)

- ☐ `docker compose up --build` starts cleanly with no errors in either container's logs.
 - ☐ `GET /healthz` returns 200 with all three providers `"closed"`.
 - ☐ `python scripts/demo_traffic.py` completes without unhandled exceptions.
 - ☐ Dashboard loads at `:8501` and all 6 panels render with data (no panel stuck on "no data").
 - ☐ `pytest` — full suite — passes with 0 failures.
 - ☐ `.env` with real keys is **not** committed to git (`git status` shows it untracked/ignored).
-

6. Known limitations to disclose (not defects)

- No streaming responses (NG2 in the PRD) — acceptable per scope.
- No admin authentication on `/v1/admin/*` (documented simplification for a student project, not a production posture).
- Free-tier providers impose their own daily/rate caps outside this system's control; if all three are simultaneously exhausted, the gateway correctly returns 502 rather than hanging — this is the system working as designed, not a bug, and should be stated as such if it happens live during a demo.